

Introduction to AI Accelerator Hardware programming

Part 2



More Control over the NVIDIA GPU

CUDA: Compute Unified Device Architecture

What is CUDA ?

- a) Parallel computing platform for NVIDIA GPUs
- b) Language extension for C, C++, and Fortran
- c) Software ecosystem
- d) All of the above

What does CUDA stand for ?

CUDA = Compute Unified Device Architecture
(not commonly spelled out anymore)

Who makes CUDA ?

NVIDIA. It's a vendor-specific system.



CUDA: Compute Unified Device Architecture

Why bother with CUDA ?

- Highly mature software
- All NVIDIA GPUs support CUDA
- The majority of GPU applications is written in CUDA
- CUDA allows low-level performance programming with reasonable productivity
- More control over the code than OpenACC



CUDA: Programming Basics

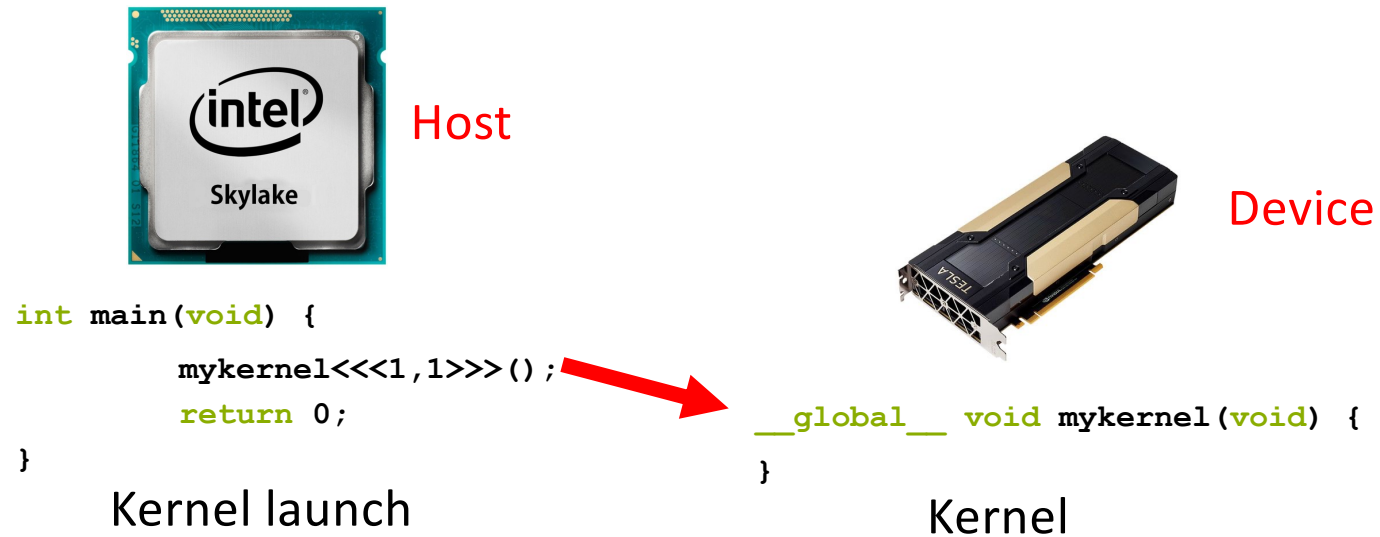
GPU `__global__ void mykernel(void) {` ← Kernel
 `}`

CPU `int main(void) {`
 `mykernel<<<1,1>>>();` ← Kernel launch
 `return 0;`
 `}`

CUDA allows writing explicit GPU functions
CUDA functions are called kernels



CUDA: Programming Basics



GPU becomes active when called upon by the CPU



Compiling CUDA Programs

```
__global__ void mykernel(void) {  
}
```

```
int main(void) {  
    mykernel<<<1,1>>>();  
    return 0;  
}
```

```
nvcc test.cu
```

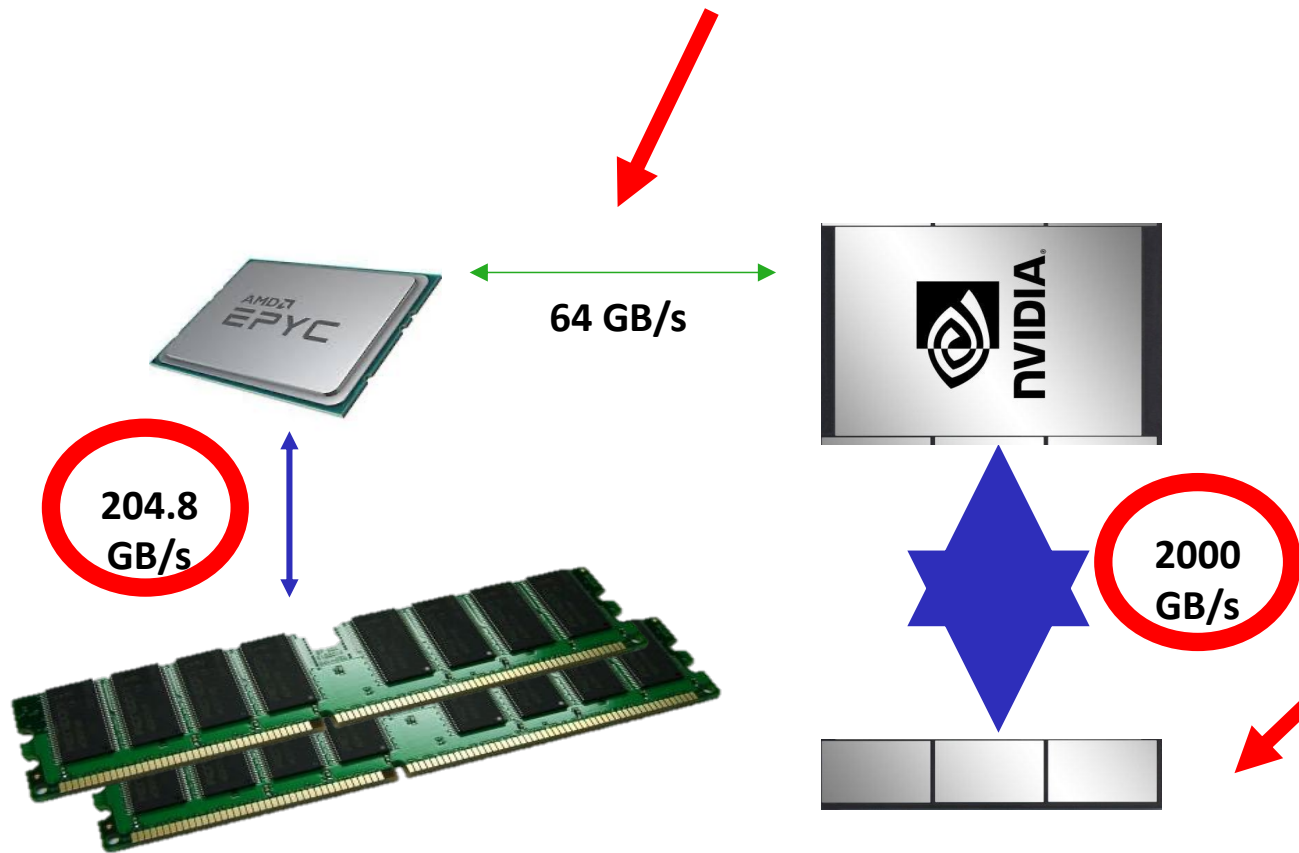
Cuda programs are compiled with nvcc
nvcc wraps a C/C++ compiler



Remember: CPU and GPU have separate memory

We need two functions to deal with the separate memories

- `cudaMemcpy`: move data between CPU and GPU



- `cudaMalloc`: allocate memory on the GPU



A first CUDA program: Listing1-12

```
#include <omp.h>
__global__ void gpusum(long steps, double* d_gpu_sum){
    double sum=0.0;
    for(int i = 0; i < steps; i++){
        sum += i;
    }
    d_gpu_sum[0]=sum;
}

int main(int argc, char *argv[]) {
    long steps = (argc > 1) ? atol(argv[1]) : 10000000000;
    double gpu_sum;
    double* d_gpu_sum;
    cudaMalloc((void **)&d_gpu_sum, sizeof(double));
    double start = omp_get_wtime();
    gpusum<<<1,1>>>(steps,d_gpu_sum);
    cudaDeviceSynchronize();
    cudaMemcpy(&gpu_sum, d_gpu_sum, sizeof(double), cudaMemcpyDeviceToHost);
    double gpu_time = omp_get_wtime() - start;
    printf("gpu sum = %.10f, steps %ld time %.3f s\n", gpu_sum, steps, gpu_time);
    return 0;
}
```

Compiling and Running the Program

```
[jla061@birget ~]$ nvcc Listing1-12-cudasum.cu -o cudasum -fopenmp
nvcc fatal   : Unknown option '-fopenmp'
```

-Xcompiler to pass arguments to the wrapped compiler

```
[jla061@birget ~]$ nvcc Listing1-12-cudasum.cu -o cudasum -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudasum 1000000000
gpu sum = 499999999067108992.0000000000,steps 1000000000  time 7.517 s
[jla061@birget ~]$ ./cudasum
gpu sum = 994142217754649344.0000000000,steps 1000000000  time 65.649 s
```

There seems to be a bug.

Compiling and Running the Program

```
[jla061@birget ~]$ nvcc Listing1-12-cudasum.cu -o cudasum -fopenmp
nvcc fatal   : Unknown option '-fopenmp'
```

-Xcompiler to pass arguments to the wrapped compiler

```
[jla061@birget ~]$ nvcc Listing1-12-cudasum.cu -o cudasum -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudasum 1000000000
gpu sum = 499999999067108992.0000000000,steps 1000000000  time 7.517 s
[jla061@birget ~]$ ./cudasum
gpu sum = 994142217754649344.0000000000,steps 1000000000  time 65.649 s
```

Remember the CPU:

```
[jla061@birget ~]$ ./cpusum
cpu sum = 4999999999067863552.0000000000,steps 1000000000  time 8.889 s
```

We are 7.4x slower than the CPU. Is that good or bad?

The diagram illustrates the GigaThread Engine with MIG Control, showing a symmetrical architecture with four main processing units. Each unit is composed of a GPC (Graphics Processing Cluster) at the top, a large L2 Cache in the center, and another GPC at the bottom. The GPCs are further divided into TPCs (Texture Processing Clusters) and SMs (Streaming Multiprocessors). The L2 Caches are connected to the GPCs via a network of interconnects. The entire engine is connected to a High-Speed Hub at the bottom, which provides NVLink interfaces. The top of the diagram shows connections to a PCI Express 4.0 Host Interface. The right side of the diagram shows connections to Memory Controllers and Memory Banks (HBM2).



You are here

More Threads: Listing1-13

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>

__global__ void gpusum(long steps, double* d_gpu_sum)
{
    long size = (steps + blockDim.x - 1) / blockDim.x;
    long start = threadIdx.x*size;
    long end = min(start+size, steps);

    double sum=0.0;
    for(long i = start; i < end; i++){
        sum += i;
    }
    d_gpu_sum[threadIdx.x]=sum;
}
```

Now each thread computes a part.

- `threadIdx.x` : id of current thread
- `blockDim.x` : number of threads



New Kernel launch and Reduction: Listing1-13

```
int main(int argc, char *argv[])
{
    long steps = (argc > 1) ? atol(argv[1]) : 10000000000;
    long threads = (argc > 2) ? atol(argv[2]) : 32;
    double* gpu_partial_sum;
    double gpu_sum;
    double* d_gpu_sum;
    gpu_partial_sum = (double *)malloc(threads * sizeof(double));
    cudaMalloc((void **)&d_gpu_sum, threads * sizeof(double));
    double start = omp_get_wtime();
    gpusum<<<1, threads>>>(steps, d_gpu_sum);
    cudaDeviceSynchronize();
    cudaMemcpy(gpu_partial_sum, d_gpu_sum, threads * sizeof(double),
               cudaMemcpyDeviceToHost);
    for(int i = 0; i < threads; i++){
        gpu_sum += gpu_partial_sum[i];
    }
    double gpu_time = omp_get_wtime() - start;
    printf("gpu sum = %.10f, steps %ld time %.3f s\n", gpu_sum, steps, gpu_time);
    return 0; 13
}
```

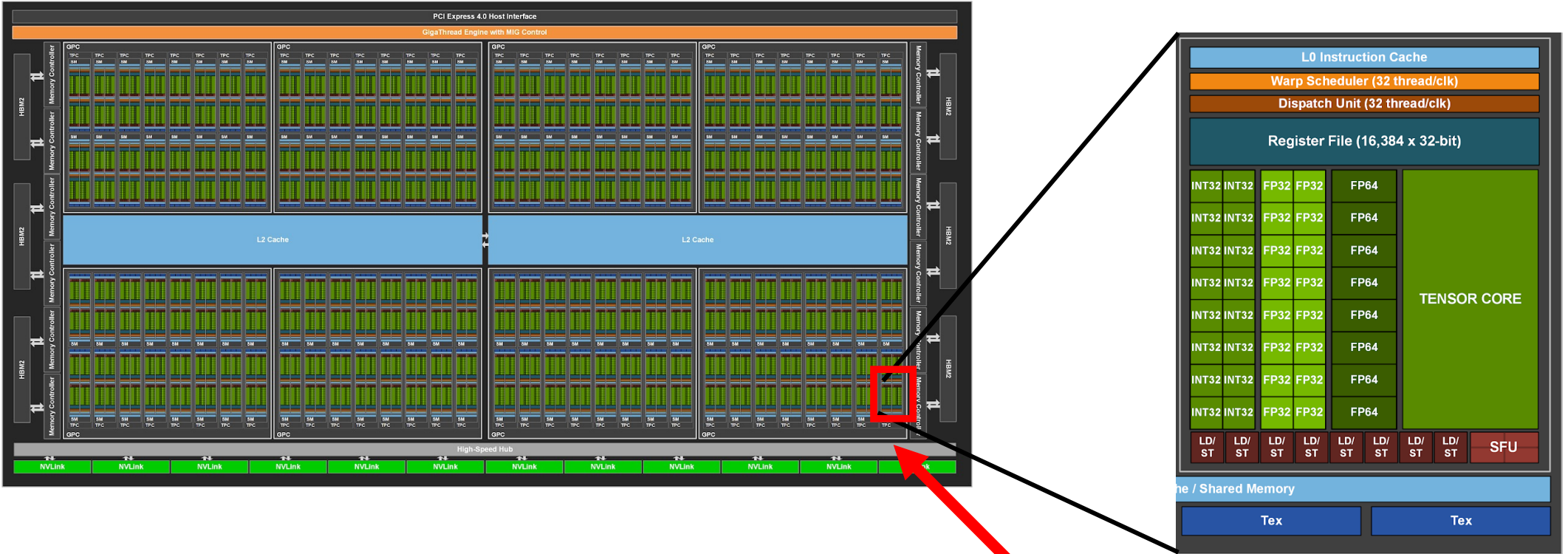
Running with more Threads

```
[jla061@birget ~]$ nvcc Listing1-13-cudasum.cu -o cudasum -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudasum 10000000000 2
gpu sum = 49999999990067462144.0000000000,steps 10000000000 time 38.820 s
[jla061@birget ~]$ ./cudasum 10000000000 4
gpu sum = 49999999990071869440.0000000000,steps 10000000000 time 19.425 s
[jla061@birget ~]$ ./cudasum 10000000000 8
gpu sum = 49999999990076293120.0000000000,steps 10000000000 time 9.722 s
[jla061@birget ~]$ ./cudasum 10000000000 16
gpu sum = 49999999990090915840.0000000000,steps 10000000000 time 4.873 s
[jla061@birget ~]$ ./cudasum 10000000000 32
gpu sum = 49999999990124470272.0000000000,steps 10000000000 time 2.477 s
[jla061@birget ~]$ ./cudasum 10000000000 64
gpu sum = 49999999990198419456.0000000000,steps 10000000000 time 1.241 s
[jla061@birget ~]$ ./cudasum 10000000000 128
gpu sum = 49999999990325837824.0000000000,steps 10000000000 time 0.640 s
[jla061@birget ~]$ ./cudasum 10000000000 256
gpu sum = 49999999990554468352.0000000000,steps 10000000000 time 0.541 s
[jla061@birget ~]$ ./cudasum 10000000000 512
gpu sum = 49999999990942244864.0000000000,steps 10000000000 time 0.554 s
```

Remember:

```
[jla061@birget ~]$ ./accsum
gpu sum = 49999999995000004608.0000000000,steps 10000000000 time 0.413 s
```

Remember the GPU Architecture again

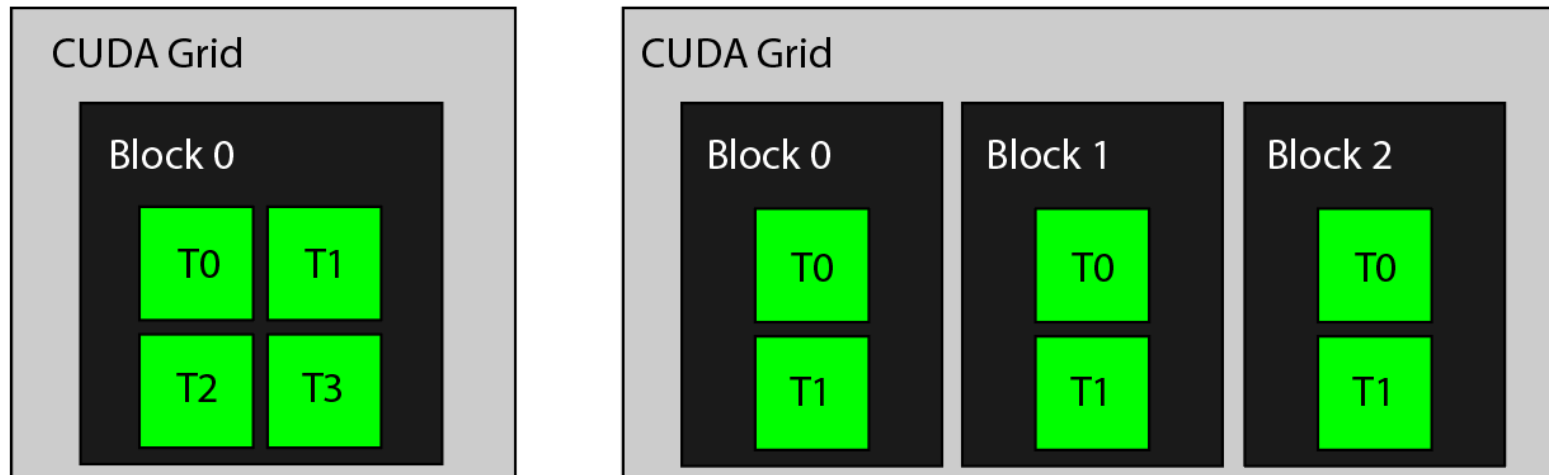


Listing1-12 uses only about 1% of the GPU

You are here

Threads vs Blocks

Why differentiate between threads and blocks ?



- Threads are located on a single multiprocessor
- Threads share fast memory
- Threads are executed together
- Threads per block are limited to 1024



Using Threads and Blocks: Listing1-14

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <math.h>
__global__ void gpusum(long steps, double* d_gpu_sum)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    long totalthreads = blockDim.x * gridDim.x;
    long size = (steps + totalthreads - 1) / totalthreads;
    long start = id*size;
    long end = min(start+size, steps);

    double sum=0.0;
    for(long i = start; i < end; i++){
        sum += i;
    }
    d_gpu_sum[id]=sum;
}
```

Now we use multiple blocks

- `blockIdx.x`: id of current block
- `gridDim.x` : number of block



Using Threads and Blocks: Listing1-14

```
int main(int argc, char *argv[])
{
    long steps = (argc > 1) ? atol(argv[1]) : 10000000000;
    long threads = (argc > 2) ? atol(argv[2]) : 32;
    long blocks = (argc > 3) ? atol(argv[3]) : 32;
    double* gpu_partial_sum;
    double gpu_sum;
    double* d_gpu_sum;
    gpu_partial_sum = (double *)malloc(blocks * threads * sizeof(double));
    cudaMalloc((void **)&d_gpu_sum, blocks * threads * sizeof(double));
    double start = omp_get_wtime();
    gpusum<<<blocks, threads>>>(steps, d_gpu_sum);
    cudaDeviceSynchronize();
    cudaMemcpy(gpu_partial_sum, d_gpu_sum, blocks * threads * sizeof(double),
               cudaMemcpyDeviceToHost);
    for(long i = 0; i < blocks*threads; i++){
        gpu_sum += gpu_partial_sum[i];
    }
    double gpu_time = omp_get_wtime() - start;
    printf("gpu18 sum = %.10f, steps %ld time %.4f s\n", gpu_sum, steps, gpu_time);
    return 0;
}
```

Running with Threads and Blocks

```
[jla061@birget ~]$ nvcc Listing1-14-cudasum.cu -o cudasum -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudasum 10000000000 512 1
gpu sum = 49999999990942244864.0000000000,steps 10000000000 time 0.5326 s
[jla061@birget ~]$ ./cudasum 10000000000 512 2
gpu sum = 49999999991561338880.0000000000,steps 10000000000 time 0.2867 s
[jla061@birget ~]$ ./cudasum 10000000000 512 4
gpu sum = 49999999992481742848.0000000000,steps 10000000000 time 0.1587 s
[jla061@birget ~]$ ./cudasum 10000000000 512 8
gpu sum = 49999999993684180992.0000000000,steps 10000000000 time 0.0812 s
[jla061@birget ~]$ ./cudasum 10000000000 512 16
gpu sum = 49999999994810572800.0000000000,steps 10000000000 time 0.0412 s
[jla061@birget ~]$ ./cudasum 10000000000 512 32
gpu sum = 49999999995000217600.0000000000,steps 10000000000 time 0.0218 s
[jla061@birget ~]$ ./cudasum 10000000000 512 64
gpu sum = 49999999995000340480.0000000000,steps 10000000000 time 0.0124 s
[jla061@birget ~]$ ./cudasum 10000000000 512 128
gpu sum = 49999999995000365056.0000000000,steps 10000000000 time 0.0133 s
[jla061@birget ~]$ ./cudasum 10000000000 512 108
gpu sum = 49999999994999963648.0000000000,steps 10000000000 time 0.0085 s
```

We are 50x faster than OpenACC

```
[jla061@birget ~]$ ./accsum
gpu sum = 49999999995000004608.0000000000,steps 10000000000 time 0.413 s
```

Remember again the GPU Architecture



108 SMs
(cores)

Best performance
with 108 Blocks

More than 10x
Faster than 2 CPUs

GA100 Full GPU with 128 SMs. The A100 Tensor Core GPU has 108 SMs.

You are here

```
[jla061@birget ~]$ numactl -C0-255 ./ompsum
cpu sum = 49999999990554484736.0000000000,steps 10000000000 time 0.096 s
[jla061@birget ~]$ ./cudasum 10000000000 512 108
gpu sum = 49999999994999963648.0000000000,steps 10000000000 time 0.0085 s
```

Now back to Vector Multiplication: Listing1-15 Part 1

```
int main(int argc, char *argv[])
{
    long n = (argc > 1) ? atol(argv[1]) : 10000000000;
    long threads = (argc > 2) ? atol(argv[2]) : 32;
    long blocks = (argc > 3) ? atol(argv[3]) : 32;
    double* gpu_partial_sum;
    double gpu_sum;
    double* d_gpu_sum;
    gpu_partial_sum = (double *)malloc(blocks * threads * sizeof(double));
    cudaMalloc((void **)&d_gpu_sum, blocks * threads * sizeof(double));
    float* A = (float*)malloc(n * sizeof(float));
    float* B = (float*)malloc(n * sizeof(float));
    float* d_A, d_B;
    cudaMalloc((void **)&d_A, n * sizeof(double));
    cudaMalloc((void **)&d_B, n * sizeof(double));
    for (long i = 0; i < n; i++) {
        A[i] = i; B[i] = 2;
    }
    cudaMemcpy(d_A, A, n * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, n * sizeof(float), cudaMemcpyHostToDevice);
    double start = omp_get_wtime();
    gpuvecmul<<<blocks, threads>>>(n, d_A, d_B, d_gpu_sum);
```



Now back to Vector Multiplication: Listing1-15 Part 2

```
__global__ void gpuvecmul(long n, float* d_A, float* d_B, double* d_gpu_sum)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    long totalthreads = blockDim.x * gridDim.x;
    long size = (n + totalthreads - 1) / totalthreads;
    long start = id*size;
    long end = min(start+size,n);

    double sum=0.0;
    for(long i = start; i < end; i++){
        sum += d_A[i]*d_B[i];
    }
    d_gpu_sum[id]=sum;
}
```



Now back to Vector Multiplication: Listing1-15 Part 3

```
gpuvecmul<<<blocks,threads>>>(n,d_A,d_B,d_gpu_sum);
cudaDeviceSynchronize();
cudaMemcpy(gpu_partial_sum, d_gpu_sum, blocks * threads * sizeof(double),
           cudaMemcpyDeviceToHost);
for(long i = 0; i < blocks*threads; i++){
    gpu_sum += gpu_partial_sum[i];
}
double gpu_time = omp_get_wtime() - start;

printf("gpu sum = %.10f, steps %ld  time %.4f s\n",gpu_sum,n,gpu_time);
free(A);
free(B);
cudaFree(d_A);
cudaFree(d_B);
return 0;
}
```



Testing the Program

```
[jla061@birget ~]$ nvcc Listing1-15-cudavecmult.cu -o cudavecmult -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudavecmult
gpu sum = 0.0000000000,steps 10000000000 time 0.3878 s
```

Something is not right. Run smaller instances:

```
[jla061@birget ~]$ ./cudavecmult 10000000000
gpu sum = 0.0000000000,steps 10000000000 time 0.3878 s
[jla061@birget ~]$ ./cudavecmult 100000000
gpu sum = 999999900000000.0000000000,steps 10000000 time 0.0049 s
[jla061@birget ~]$ ./cudavecmult 100000000
gpu sum = 99999999000000000.0000000000,steps 100000000 time 0.0148 s
[jla061@birget ~]$ ./cudavecmult 1000000000
gpu sum = 9999999990000000000.0000000000,steps 1000000000 time 0.1274 s
[jla061@birget ~]$ ./cudavecmult 10000000000
gpu sum = 0.0000000000,steps 10000000000 time 0.3464 s
```

Seems to be correct, except for the largest instance. Why ?

Error Handling: Listing1-16

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <iostream>

#define CHECK_CUDA_ERROR(val) check((val), #val, __FILE__, __LINE__)
void check(cudaError_t err, char const* func, char const* file, int line)
{
    if (err != cudaSuccess)
    {
        std::cout << "CUDA Runtime Error at: " << file << ":" << line
                    << std::endl;
        std::cout << cudaGetErrorString(err) << " " << func << std::endl;
        std::exit(EXIT_FAILURE);
    }
}

int main(int argc, char *argv[])
{
    ...
    CHECK_CUDA_ERROR(cudaMalloc((void **)&d_A, n * sizeof(double)));
    CHECK_CUDA_ERROR(cudaMalloc((void **)&d_B, n * sizeof(double)));
}
```



With Error Handling

```
[jla061@birget ~]$ nvcc Listing1-16-cudavecmult.cu -o cudavecmult -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudavecmult
CUDA Runtime Error at: Listing1-16-cudavecmult.cu:53
out of memory cudaMalloc((void **)&d_B,n * sizeof(double))
```

Now OOM error is caught. Remember: the A100 has “only” 80GB

```
[jla061@birget ~]$ ./cudavecmult 1000000000
gpu sum = 999999999000000000.000000000,steps 1000000000 time 0.1202 s
[jla061@birget ~]$ ./cudavecmult 100000000
gpu sum = 9999999900000000.000000000,steps 100000000 time 0.0155 s
```

Smaller instances run correctly.

How much memory do we need?

$2 \text{ (arrays)} * 4 \text{ (float)} * 10^{10} = 80\text{GB}$

Why does it not fit?

One final Correction w.r.t. Memory: Listing1-17

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <iostream>

#define CHECK_CUDA_ERROR(val) check((val), #val, __FILE__, __LINE__)
void check(cudaError_t err, char const* func, char const* file, int line)
{
    if (err != cudaSuccess)
    {
        std::cout << "CUDA Runtime Error at: " << file << ":" << line
                    << std::endl;
        std::cout << cudaGetErrorString(err) << " " << func << std::endl;
        std::exit(EXIT_FAILURE);
    }
}

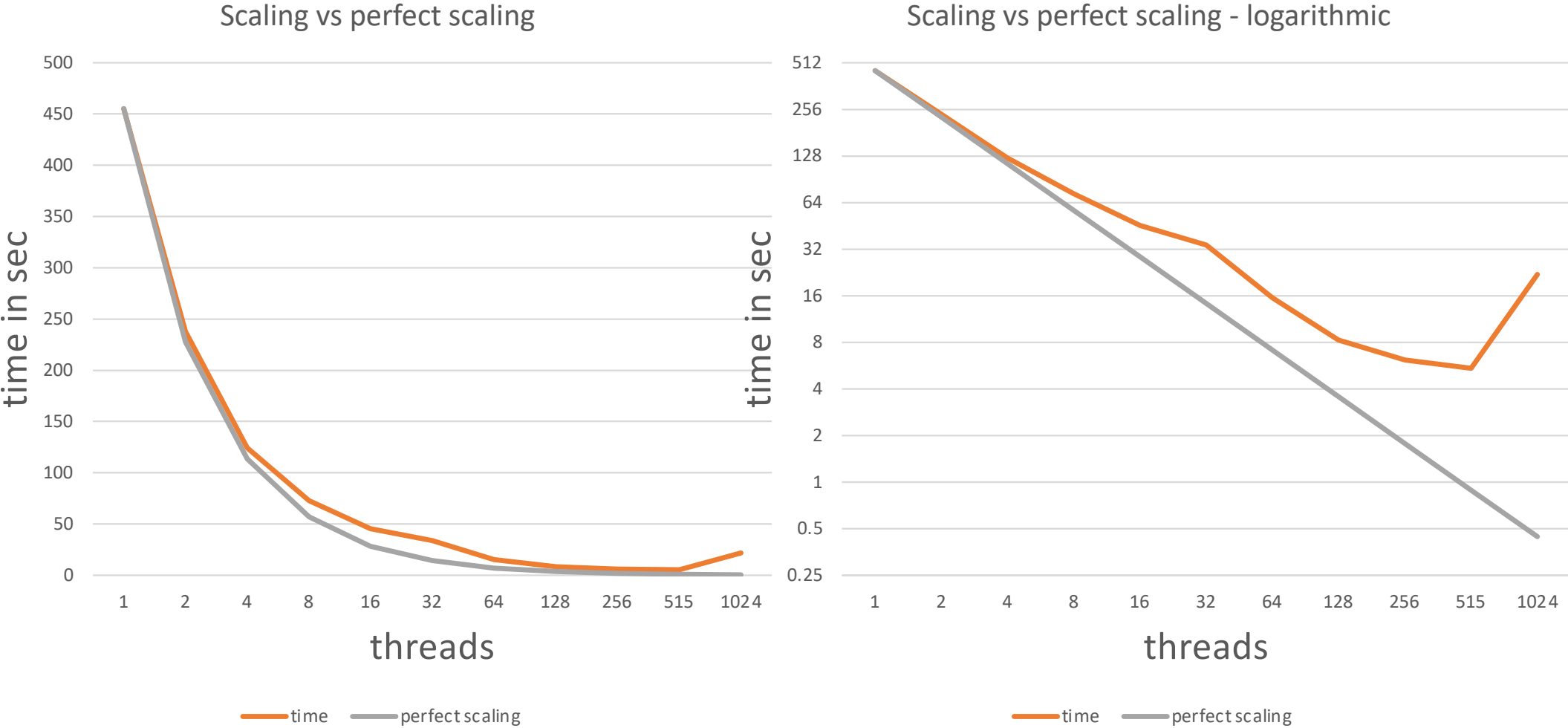
int main(int argc, char *argv[])
{
    ...
    CHECK_CUDA_ERROR(cudaMalloc((void **)&d_A, n * sizeof(float)));
    CHECK_CUDA_ERROR(cudaMalloc((void **)&d_B, n * sizeof(float)));
}
```



Benchmarking the final Program: Thread scaling

```
[jla061@birget ~]$ nvcc Listing1-17-cudavecmult.cu -o cudavecmult -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudavecmult 10000000000 1 1
gpu sum = 99999999981510836224.0000000000,steps 10000000000 time 455.2142 s
[jla061@birget ~]$ ./cudavecmult 10000000000 2 1
gpu sum = 99999999982369996800.0000000000,steps 10000000000 time 237.9645 s
[jla061@birget ~]$ ./cudavecmult 10000000000 4 1
gpu sum = 99999999983808479232.0000000000,steps 10000000000 time 124.1233 s
[jla061@birget ~]$ ./cudavecmult 10000000000 8 1
gpu sum = 99999999987364102144.0000000000,steps 10000000000 time 72.8865 s
[jla061@birget ~]$ ./cudavecmult 10000000000 16 1
gpu sum = 99999999989966487552.0000000000,steps 10000000000 time 45.4030 s
[jla061@birget ~]$ ./cudavecmult 10000000000 32 1
gpu sum = 99999999989999960064.0000000000,steps 10000000000 time 33.9229 s
[jla061@birget ~]$ ./cudavecmult 10000000000 64 1
gpu sum = 99999999989999927296.0000000000,steps 10000000000 time 15.4771 s
[jla061@birget ~]$ ./cudavecmult 10000000000 128 1
gpu sum = 99999999990000009216.0000000000,steps 10000000000 time 8.2587 s
[jla061@birget ~]$ ./cudavecmult 10000000000 256 1
gpu sum = 99999999989999960064.0000000000,steps 10000000000 time 6.1714 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 1
gpu sum = 99999999989999976448.0000000000,steps 10000000000 time 5.4380 s
[jla061@birget ~]$ ./cudavecmult 10000000000 1024 1
qpu sum = 99999999989999992832.0000000000,steps 10000000000 time 21.9005 s
```

Benchmarking the final Program: Thread scaling

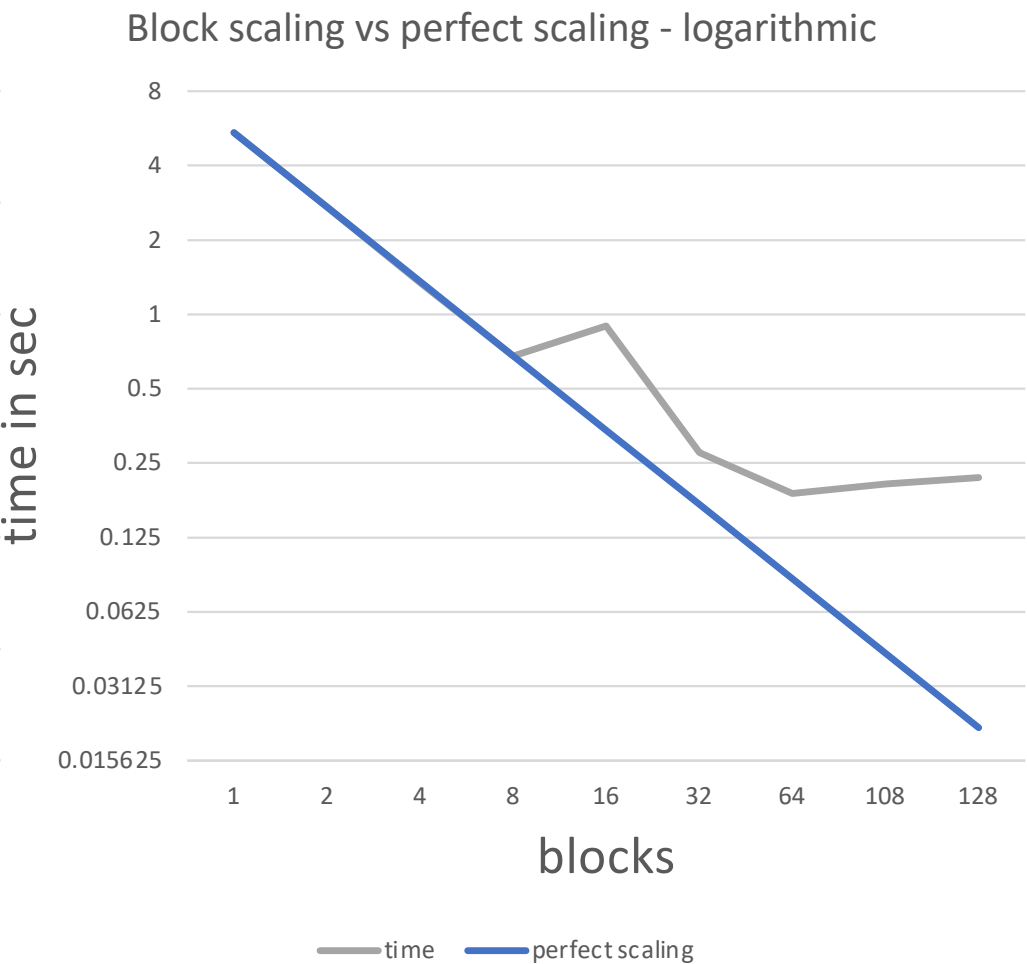
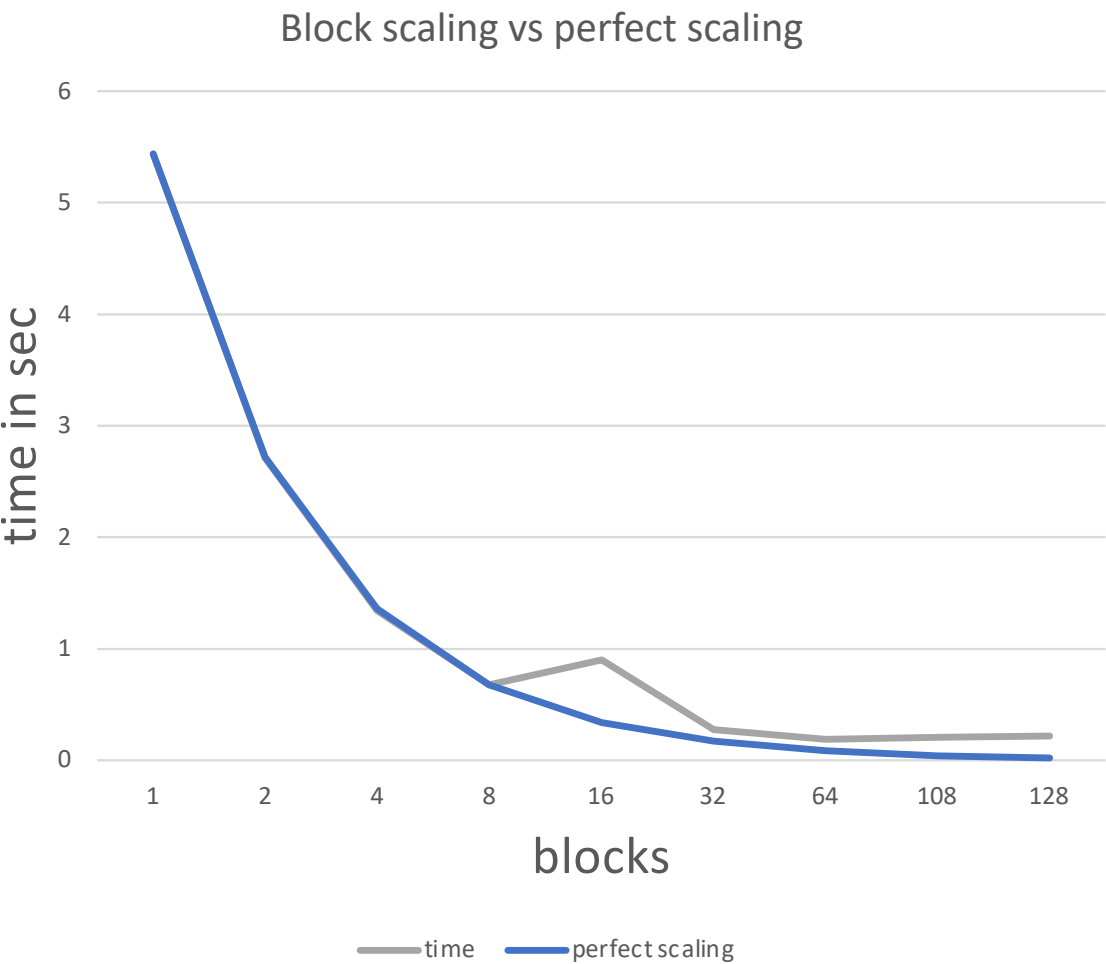


Benchmarking the final Program: Block scaling

```
[jla061@birget ~]$ nvcc Listing1-17-cudavecmult.cu -o cudavecmult -Xcompiler -fopenmp
[jla061@birget ~]$ ./cudavecmult 10000000000 512 1
gpu sum = 99999999989999976448.0000000000,steps 10000000000 time 5.4380 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 2
gpu sum = 99999999989999992832.0000000000,steps 10000000000 time 2.7152 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 4
gpu sum = 99999999989999992832.0000000000,steps 10000000000 time 1.3442 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 8
gpu sum = 9999999999000222208.0000000000,steps 10000000000 time 0.6796 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 16
gpu sum = 99999999990008348672.0000000000,steps 10000000000 time 0.8993 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 32
gpu sum = 99999999990023847936.0000000000,steps 10000000000 time 0.2754 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 64
gpu sum = 99999999989999992832.0000000000,steps 10000000000 time 0.1884 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 128
gpu sum = 99999999990005022720.0000000000,steps 10000000000 time 0.2179 s
[jla061@birget ~]$ ./cudavecmult 10000000000 512 108
gpu sum = 99999999989998338048.0000000000,steps 10000000000 time 0.2054 s
[jla061@birget ~]$ ./cudavecmult 10000000000 128 108
gpu sum = 99999999989997223936.0000000000,steps 10000000000 time 0.1365 s
[jla061@birget ~]$ numactl -C0-127 ./ompvectormult 10000000000
cpu sum = 99999999989999992832.0000000000,elements 10000000000 time 0.729 s
```

About 5x
Correct?

Benchmarking the final Program: Block scaling



Reduction in CUDA: Listing1-17

```
cudaMemcpy(gpu_partial_sum, d_gpu_sum, blocks * threads * sizeof(double),  
           cudaMemcpyDeviceToHost);  
for(long i = 0; i < blocks*threads; i++){  
    gpu_sum += gpu_partial_sum[i];  
}
```

In parallel computing, a **reduction** efficiently combines elements from a large dataset (like an array) into a single result according to some operation.

Typical operations:

- Sum
- Maximum
- Minimum



Reduction in CUDA: Listing1-17

```
cudaMemcpy(gpu_partial_sum, d_gpu_sum, blocks * threads * sizeof(double),  
           cudaMemcpyDeviceToHost);  
for(long i = 0; i < blocks*threads; i++){  
    gpu_sum += gpu_partial_sum[i];  
}
```

The reduction is performed sequentially on the CPU.
Is it efficient? Lets time it.



Timing the Reduction: Listing2-1

```
double start = omp_get_wtime();
gpuvecmul<<<blocks,threads>>>(n,d_A,d_B,d_gpu_sum);
cudaDeviceSynchronize();
double gpu_time = omp_get_wtime() - start;
start = omp_get_wtime();
cudaMemcpy(gpu_partial_sum, d_gpu_sum, blocks * threads * sizeof(double),
           cudaMemcpyDeviceToHost);
for(long i = 0; i < blocks*threads; i++){
    gpu_sum += gpu_partial_sum[i];
}
double reducetime = omp_get_wtime() - start;

printf("gpu sum = %.10f, steps %ld compute time %.4f s reduction time %.4f s\n",
       gpu_sum, n, gpu_time, reducetime);
```

Now we will see how much time it takes.



Timing the Reduction: Listing2-1

```
[jla061@birget ~]$ nvcc Listing2-1-cudavecmult.cu -o cudavecmult -Xcompiler -fopenmp  
[jla061@birget ~]$ ./cudavecmult 10000000000 512 108  
gpu sum = 99999999989998338048.0000000000, steps 10000000000 compute time 0.2079 s  
reduction time 0.0005 s
```

Takes very little time. But what if we need to perform the reduction on the GPU?



Simple CUDA Reduction Kernel: Listing2-2

```
__global__ void reduction(int blocks, int threads, double* d_final_gpu_sum, double*  
d_gpu_sum)  
{  
    for(long i = 0; i < blocks*threads; i++){  
        d_final_gpu_sum[0] += d_gpu_sum[i];  
    }  
}
```

```
int main(int argc, char *argv[])  
{
```

```
    ...  
    double start = omp_get_wtime();  
    gpuvecmul<<<blocks, threads>>>(n, d_A, d_B, d_gpu_sum);  
    cudaDeviceSynchronize();  
    double gpu_time = omp_get_wtime() - start;  
    start = omp_get_wtime();  
    reduction<<<1, 1>>>(blocks, threads, d_final_gpu_sum, d_gpu_sum);  
    cudaDeviceSynchronize();  
    cudaMemcpy(&gpu_sum, d_final_gpu_sum, sizeof(double), cudaMemcpyDeviceToHost);  
    double reducetime = omp_get_wtime() - start;  
    printf("gpu sum = %.10f, steps %ld compute time %.4f s reduction time %.5f s  
        \n", gpu_sum, n, gpu_time, reducetime);
```

Do we need this?
Why?



Timing the CUDA Reduction Kernel: Listing2-2

```
[jla061@birget ~]$ nvcc Listing2-2-reductionvecmult.cu -o reductionvecmult -Xcompiler -fopenmp
```

```
[jla061@birget ~]$ ./reductionvecmult 10000000000 512 108
```

```
gpu sum = 99999999989998338048.0000000000, steps 10000000000 compute time 0.2075 s  
reduction time 0.00539 s
```

```
[jla061@birget ~]$ ./reductionvecmult 1000000 512 108
```

```
gpu sum = 999999000000.0000000000, steps 1000000 compute time 0.0069 s reduction time  
0.00494 s
```

As expected, takes about 10x longer on the GPU.

Even worse when the vector size is small.

We can improve upon this by using parallelism.



Parallel CUDA Reduction Kernel: Listing2-3

```
__global__ void gpuvecmul(long n, float* d_A, float* d_B, double* d_gpu_partial_sum,
double* d_gpu_sum)
{
```

```
    ...
    for(long i = start; i < end; i++){
        sum += d_A[i]*d_B[i];
    }
    d_gpu_partial_sum[threadIdx.x]=sum;
    __syncthreads();
    if(threadIdx.x==0)
    for(int j = 0; j < blockDim.x; j++){
        d_gpu_sum[blockIdx.x] += d_gpu_partial_sum[j];
    }
}
```

Make sure all
threads in block
are done

```
__global__ void reduction(int blocks, double* d_final_gpu_sum, double* d_gpu_sum)
{
    for(long i = 0; i < blocks; i++){
        d_final_gpu_sum[0] += d_gpu_sum[i];
    }
}
```



Testing the CUDA Reduction Kernel: Listing2-3

```
[jla061@birget ~]$ nvcc Listing2-3-reductionvecmult.cu -o reductionvecmult -Xcompiler -fopenmp
[jla061@birget ~]$ ./reductionvecmult 10000000000 512 108
gpu sum = 99999999990000025600.0000000000,steps 10000000000 compute time 0.2097 s
reduction time 0.00011 s
[jla061@birget ~]$ ./reductionvecmult 10000000000 512 108
gpu sum = 99999999990000025600.0000000000,steps 10000000000 compute time 0.2109 s
reduction time 0.00013 s

[jla061@birget ~]$ ./reductionvecmult 100000000 512 108
gpu sum = 9999999900000000.0000000000,steps 100000000 compute time 0.0042 s reduction
time 0.00010 s
```

Reduction is fast now, but we cheated!

Part of the reduction is in the `gpuvecmul` kernel.

Since timings are not fully stable, we need to move the reduction out.



Timing the CUDA Reduction Kernel: Listing2-4

```
__global__ void gpuvecmul(long n, float* d_A, float* d_B, double* d_gpu_partial_sum,
double* d_gpu_sum)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    long totalthreads = blockDim.x * gridDim.x;
    long size = (n + totalthreads - 1) / totalthreads;
    long start = id*size;
    long end = min(start+size,n);

    double sum=0.0;
    for(long i = start; i < end; i++){
        sum += d_A[i]*d_B[i];
    }
    d_gpu_partial_sum[id]=sum;
}
```

__syncthreads();
Not needed here

```
int main(int argc, char *argv[])
{
    ...
    start = omp_get_wtime();
    reduction<<<blocks,threads>>>(n,d_gpu_partial_sum,d_gpu_sum);
    reductionFinish<<<1,1>>>(blocks,d_final_gpu_sum,d_gpu_sum);
    cudaDeviceSynchronize();
}
```



Timing the CUDA Reduction Kernel: Listing2-4

```
__global__ void reduction(long n, double* d_gpu_partial_sum, double* d_gpu_sum)
{
    if(threadIdx.x==0) {
        d_gpu_sum[blockIdx.x] = 0.0;
        for(int j = 0; j < blockDim.x; j++){
            d_gpu_sum[blockIdx.x] += d_gpu_partial_sum[
                blockIdx.x*blockDim.x+j];
        }
    }
}
```

same block size but
only one thread
working

```
__global__ void reductionFinish(int blocks, double* d_final_gpu_sum, double*
d_gpu_sum)
{
    for(long i = 0; i < blocks; i++){
        d_final_gpu_sum[0] += d_gpu_sum[i];
    }
}
```

Can we do this in one kernel?



Are we fast yet? Performance analysis.

- We have 10^{10} “steps” (vector length)
- In `sum += d_A[i]*d_B[i];`
we perform a multiplication and an addition
- So, we perform 2 times 10^{10} FLOPS in about 0.2 sec =
100 GFLOPS in single precision
- Let`s check the specs



Are we fast yet? Performance analysis.

Table 1. NVIDIA A100 Tensor Core GPU Performance Specs

Peak FP64 ¹	9.7 TFLOPS
Peak FP64 Tensor Core ¹	19.5 TFLOPS
Peak FP32 ¹	<u>19.5 TFLOPS</u>
Peak FP16 ¹	78 TFLOPS
Peak BF16 ¹	39 TFLOPS
Peak TF32 Tensor Core ¹	156 TFLOPS 312 TFLOPS ²
Peak FP16 Tensor Core ¹	312 TFLOPS 624 TFLOPS ²
Peak BF16 Tensor Core ¹	312 TFLOPS 624 TFLOPS ²
Peak INT8 Tensor Core ¹	624 TOPS 1,248 TOPS ²
Peak INT4 Tensor Core ¹	1,248 TOPS 2,496 TOPS ²

1 - Peak rates are based on GPU Boost Clock.

2 - Effective TFLOPS / TOPS using the new Sparsity feature

<https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>



Are we fast yet? Performance analysis.

- 100 out of 19,500 GFLOPS = less than 0.5%
- Are we being cheated?
- Where is the performance?



Performance analysis: Memory Bandwidth

- We have 10^{10} “steps”
- In `sum += d_A[i]*d_B[i];`
we load two float values, i.e. 8 bytes
- So we load 8 times 10^{10} byte = 80 GB in about
0.2 sec = 400 GB/s
- Let`s check the specs
- `nvidia-smi` : shows the available GPUs

```
[jla061@birget ~]$ nvidia-smi  
Sun Jan 25 23:03:45 2026
```

NVIDIA-SMI 590.48.01			Driver Version: 590.48.01			CUDA Version: 13.1		
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.		
						MIG M.		
0	NVIDIA A100-SXM4-80GB	Off	00000000:07:00.0	Off		0		
N/A	24C	P0	59W / 400W	0MiB / 81920MiB	0%	Default Disabled		



Performance analysis: Memory Bandwidth

NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

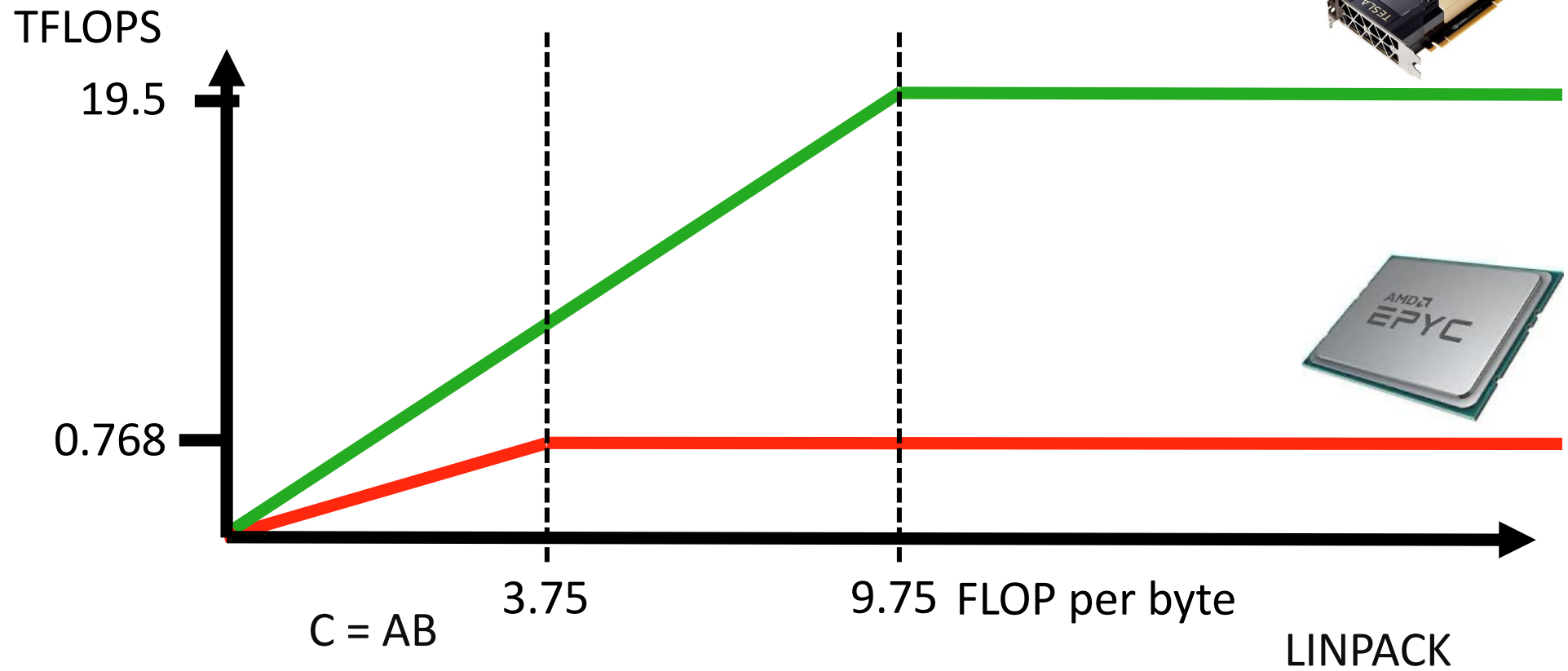
	A100 80GB PCIe	A100 80GB SXM
FP64	9.7 TFLOPS	
FP64 Tensor Core	19.5 TFLOPS	
FP32	19.5 TFLOPS	
Tensor Float 32 [TF32]	156 TFLOPS 312 TFLOPS*	
BFLOAT16 Tensor Core	312 TFLOPS 624 TFLOPS*	
FP16 Tensor Core	312 TFLOPS 624 TFLOPS*	
INT8 Tensor Core	624 TOPS 1248 TOPS*	
GPU Memory	80GB HBM2e	80GB HBM2e
GPU Memory Bandwidth	1,935GB/s	2,039GB/s

- Upper limit is about 2 TB/s
- Actual upper limit is about 1.6 TB/s
- So we are at about 25% maximum.
- Maybe we can improve a bit.



<https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>

Roofline Model to the Rescue



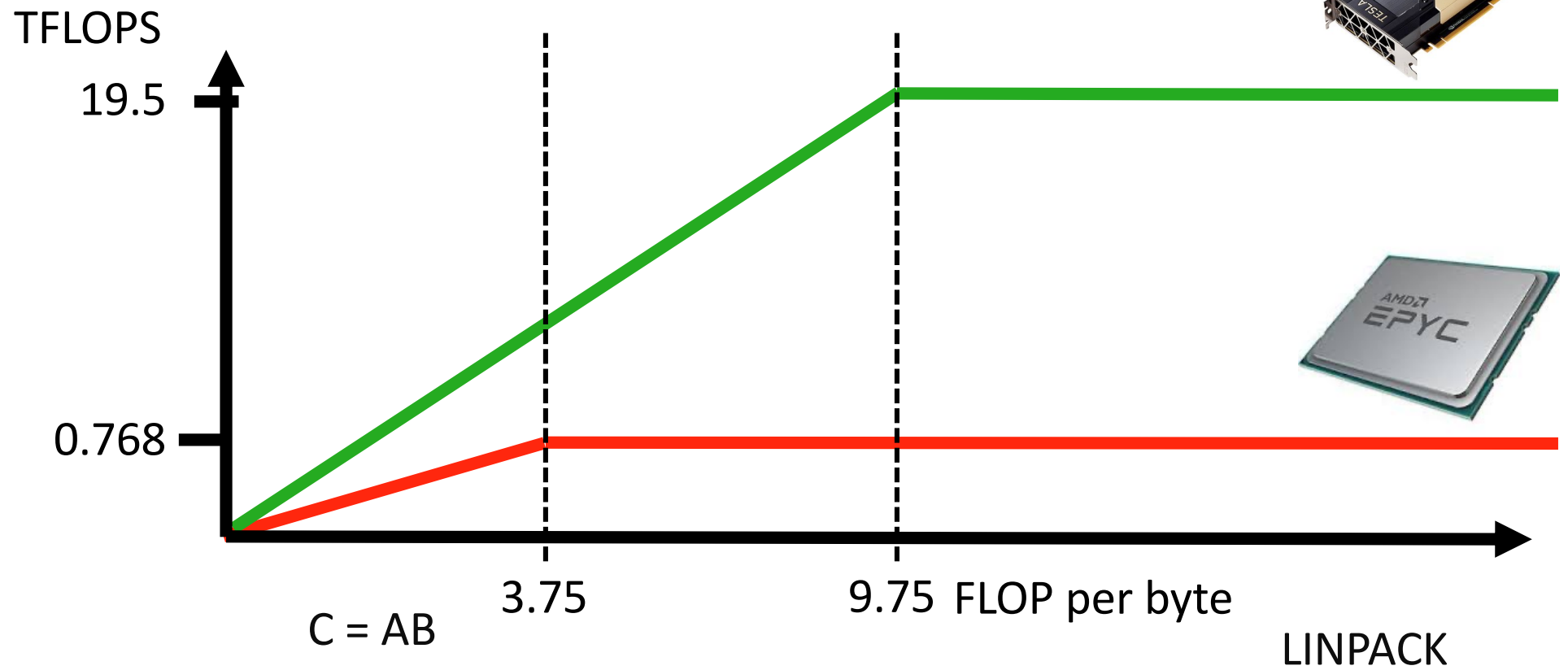
AMD EPYC 7302p

$$768 / 204.8 = 3.75$$

NVIDIA Ampere A100. 19500 / 2000 = 9.75

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65-76.

Roofline Model to the Rescue

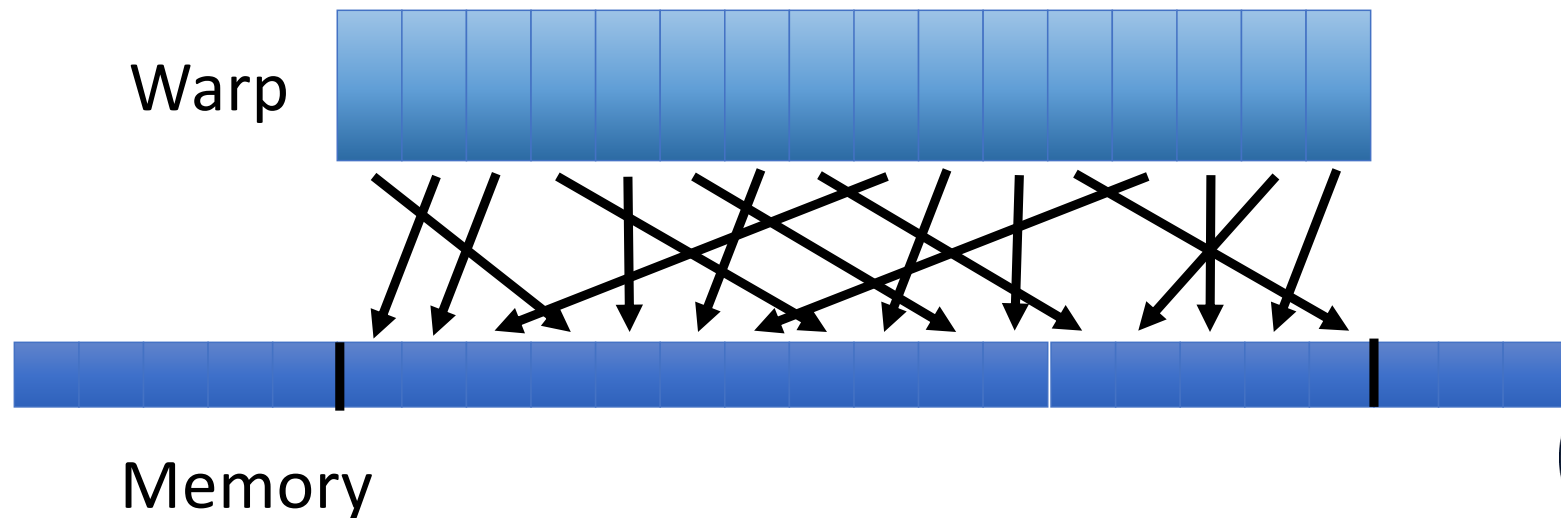


The Roofline model predicts the maximally attainable performance given the arithmetic intensity (FLOP per byte) of a code.

<https://people.eecs.berkeley.edu/~kubitron/cs252/handouts/papers/RooflineVyNoYellow.pdf>

Why don't we get the full Memory Bandwidth

A warp of 32 threads must read 32 contiguous elements from an array to get the maximum memory bandwidth, although elements can be swapped between the threads.

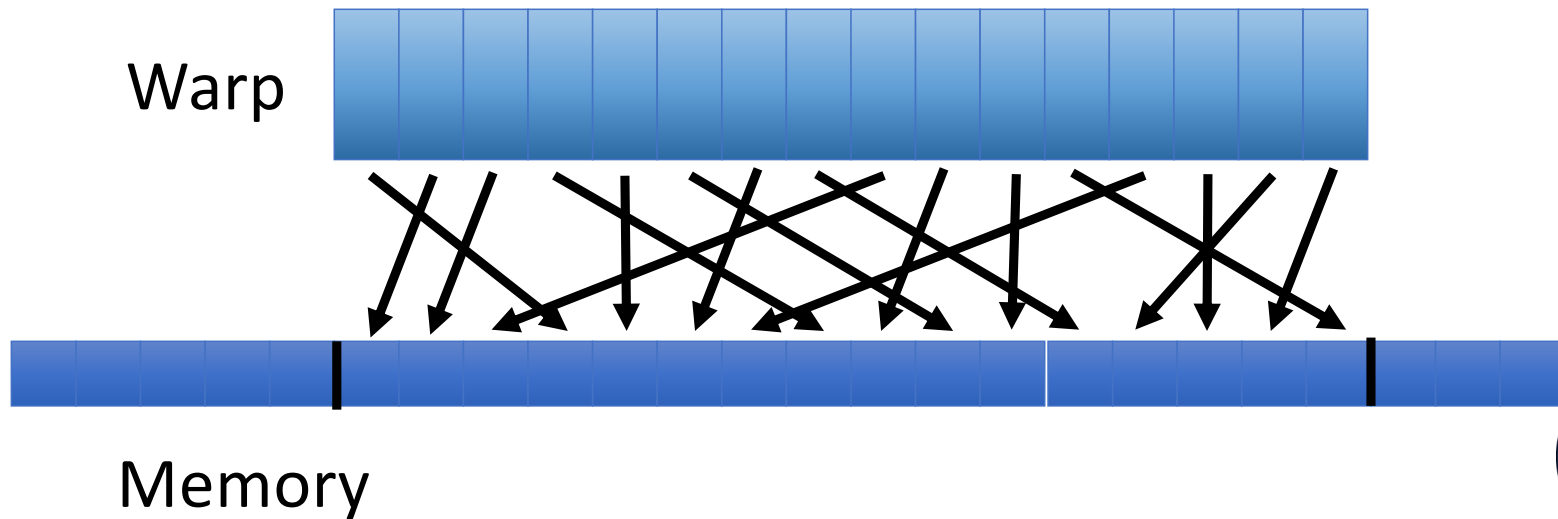


Memory Coalescing

Remember the code:

```
for(long i = start; i < end; i++){  
    sum += d_A[i]*d_B[i];  
}
```

This does not access the memory in a coalesced fashion.



Very Simple Vector Multiplication: Listing2-5

```
__global__ void puregpuvecmul(long n, float* d_A, float* d_B, float* d_C)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    int sum;
    if(id<n)
        d_C[id] = d_A[id]*d_B[id];
}

int main(int argc, char *argv[])
{
    long n = (argc > 1) ? atol(argv[1]) : 10000000000;
    long threads_per_block = (argc > 2) ? atol(argv[2]) : 32;
    int blocks = ceil(n / threads_per_block);
    ...
    puregpuvecmul<<<blocks, threads_per_block>>>(n, d_A, d_B, d_C);
    cudaDeviceSynchronize();
    double gpu_time = omp_get_wtime() - start;
    start = omp_get_wtime();
    cudaMemcpy(C, d_C, n * sizeof(float), cudaMemcpyDeviceToHost);
    for (long i = 0; i < n; i++) {
        gpu_sum+=C[i];
    }
    double reducetime = omp_get_wtime() - start;
```

Now: n total
threads



Very Simple Vector Multiplication Test: Listing2-5

```
[jla061@birget ~]$ nvcc Listing2-5-fastmultvec.cu -o fastmultvec -Xcompiler -fopenmp
[jla061@birget ~]$ ./fastmultvec 10000000000 32
CUDA Runtime Error at: Listing2-5-fastmultvec.cu:40
out of memory cudaMalloc((void **)&d_C,n * sizeof(float))
[jla061@birget ~]$ ./fastmultvec 5000000000 32
gpu sum = 18446744066638389248.0000000000,steps 5000000000 compute time 0.1178 s copy
time 27.05050 s
[jla061@birget ~]$ ./fastmultvec 5000000000 64
gpu sum = 18446744066638389248.0000000000,steps 5000000000 compute time 0.0612 s copy
time 26.85382 s
[jla061@birget ~]$ ./fastmultvec 5000000000 128
gpu sum = 18446744066638389248.0000000000,steps 5000000000 compute time 0.0386 s copy
time 26.96740 s
```

Now we write to GPU memory

Third array takes space, need to run a smaller problem

Too few threads = we don't get maximum memory bandwidth

Always use 128+



Very Simple Vector Multiplication Test: Listing2-5

```
[jla061@birget ~]$ ./fastmultvec 5000000000 256
gpu sum = 18446744066638389248.0000000000,steps 5000000000 compute time 0.0380 s copy
time 27.28055 s
[jla061@birget ~]$ ./fastmultvec 5000000000 512
gpu sum = 18446744066638389248.0000000000,steps 5000000000 compute time 0.0385 s copy
time 26.87311 s
[jla061@birget ~]$ ./fastmultvec 5000000000 1024
gpu sum = 18446744066638389248.0000000000,steps 5000000000 compute time 0.0389 s copy
time 26.86199 s
[jla061@birget ~]$ ./fastmultvec 5000000000 2048
gpu sum = 0.0000000000,steps 5000000000 compute time 0.0022 s copy time 27.13169 s
```

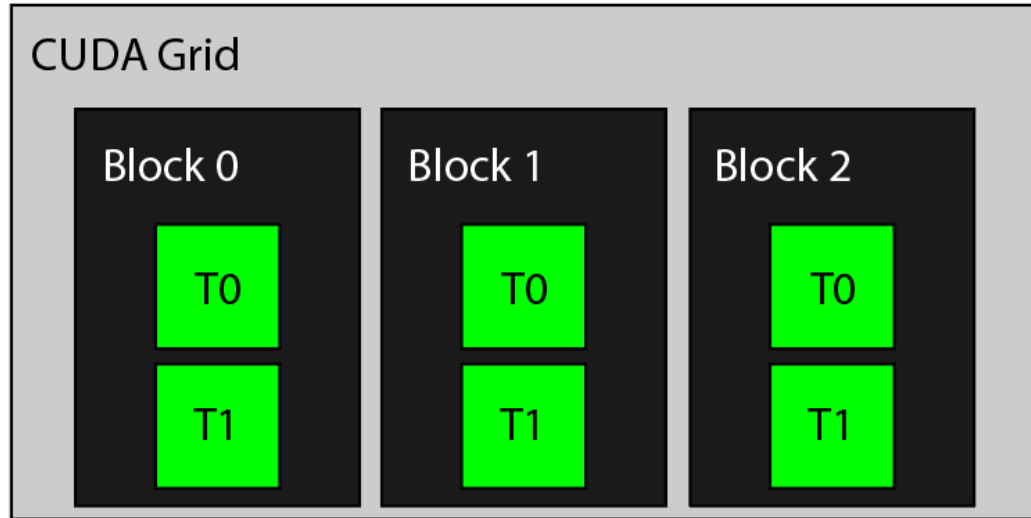
No change for the larger thread block sizes

Maximum thread block size is 1024.

Setting it to 2048 leads to incorrect results.



Threads vs Blocks vs Grid – why the difference?



- A group of threads is called a *thread block*
- A *thread block* is executed together on an SM
- We use n threads
- So we need $n/\text{thread block size}$ blocks
- There is usually a partial block at the end
- Up to 2048 threads can be active on an SM



Are we fast yet? Performance analysis again.

- We have $5 \cdot 10^9$ “steps”
- In `d_C[id] = d_A[id]*d_B[id];`
we load two float values, i.e. 8 bytes
- So we load 8 times $5 \cdot 10^9$ byte = 40 GB
- We also write 4 times $5 \cdot 10^9$ byte = 20 GB
- So we move 60 GB in about 0.0380 sec = 1,578 GB/s
- Actual upper limit is about 1.6 TB/s
- We reached the “Speed of light”
- But now we need a fast reduction. Using the existing kernels won’t do.



Reduction in Cuda

We launched at least n threads here

```
int blocks = ceil(n / threads_per_block);  
...  
puregpuvecmul<<<blocks, threads_per_block>>>(n, d_A, d_B, d_C);
```

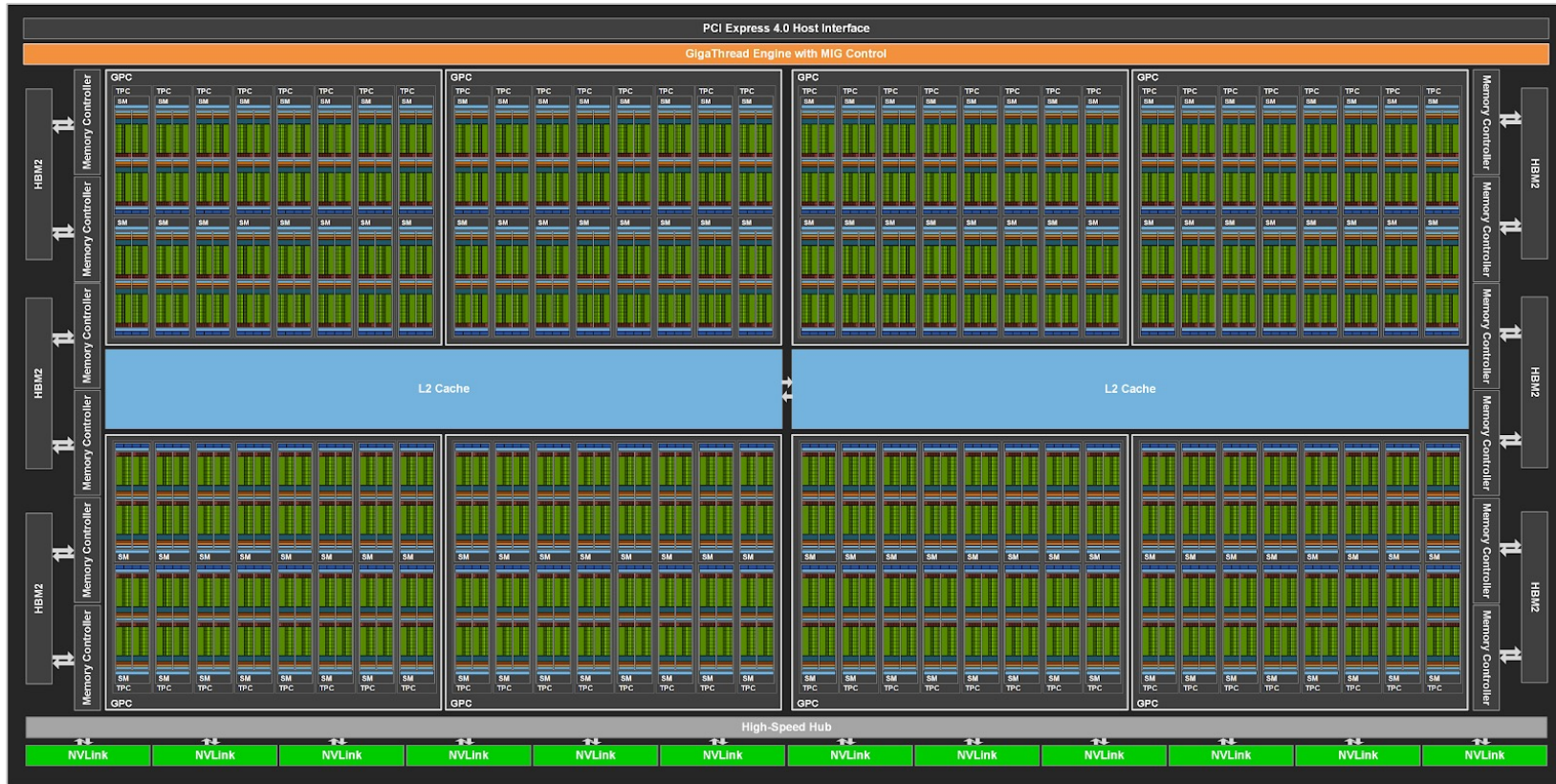
Can't do a reduction when every thread has only one value.

We need something else.

But it is not easy to communicate between threads since not all threads exist at the same time.



The A100 Ampere GPU again



108 SMs
(cores)

2048 threads per
SM

218,592 active
threads

But $n = 5 \cdot 10^9$

GA100 Full GPU with 128 SMs. The A100 Tensor Core GPU has 108 SMs.

Not all threads are running at the same time

The GPU schedules threads when SMs become available

A block is scheduled together – but we can't talk to threads in other blocks directly!

Faster Reduction in CUDA: Listing2-6

```
__global__ void reduction(long n,int gridsize,float* d_C, double* d_gpu_partial_sum,
double* d_gpu_sum)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    double sum = 0.0;
    for(long element = id; element < n; element += gridsize) {
        sum += d_C[element];
    }
    d_gpu_partial_sum[id]=sum;
    __syncthreads();

    if(threadIdx.x==0) {
        d_gpu_sum[blockIdx.x] = 0.0;
        for(int j = 0; j < blockDim.x; j++){
            d_gpu_sum[blockIdx.x] +=
                d_gpu_partial_sum[ blockIdx.x*blockDim.x+j];
        }
    }
}
```



Faster Reduction in CUDA: Listing2-6

```
__global__ void reductionFinish(int blocks, double* d_final_gpu_sum, double* d_gpu_sum)
{
    for(long i = 0; i < blocks; i++){
        d_final_gpu_sum[0] += d_gpu_sum[i];
    }
}

int main(int argc, char *argv[])
{
    long n = (argc > 1) ? atol(argv[1]) : 10000000000;
    long threads_per_block = (argc > 2) ? atol(argv[2]) : 32;
    double gpu_sum=0.0;

    long threads = (argc > 3) ? atol(argv[2]) : 256;
    long reductionblocks = (argc > 4) ? atol(argv[3]) : 108;

    double* d_final_gpu_sum;
    double* d_gpu_partial_sum;
    double* d_gpu_sum;
    cudaMalloc((void **)&d_gpu_partial_sum, reductionblocks * threads * sizeof(double));
    cudaMalloc((void **)&d_gpu_sum, reductionblocks * sizeof(double));
    cudaMalloc((void **)&d_final_gpu_sum, sizeof(double));
}
```



Faster Reduction in CUDA: Listing2-6

```
start = omp_get_wtime();  
reduction<<<reductionblocks,threads>>>(n,gridsize,d_C,d_gpu_partial_sum,d_gpu_sum);  
cudaDeviceSynchronize();  
reductionFinish<<<1,1>>>(reductionblocks,d_final_gpu_sum,d_gpu_sum);  
cudaMemcpy(&gpu_sum, d_final_gpu_sum, sizeof(double), cudaMemcpyDeviceToHost);  
cudaDeviceSynchronize();  
double reducetime = omp_get_wtime() - start;
```



Testing a faster Reduction in CUDA: Listing2-6

```
[jla061@birget Listings2]$ nvcc Listing2-6-fastreductionvecmult.cu -o
fastreductionvecmult -Xcompiler -fopenmp
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000
gpu sum = 18446744069414598656.0000000000,steps 5000000000 compute time 0.1194 s
reduction time 0.02840 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024
gpu sum = 18446744069414598656.0000000000,steps 5000000000 compute time 0.0394 s
reduction time 0.02848 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024 128
gpu sum = 18446744069414594560.0000000000,steps 5000000000 compute time 0.0394 s
reduction time 0.01336 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024 64
gpu sum = 18446744069414594560.0000000000,steps 5000000000 compute time 0.0391 s
reduction time 0.01316 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024 64 64
gpu sum = 18446744069414584320.0000000000,steps 5000000000 compute time 0.0394 s
reduction time 0.01683 s
```

Reduction still a bit slow since we need to read the data again.



Faster Reduction in CUDA: Listing2-6

```
__global__ void reduction(long n, int gridsize, float* d_C, double* d_gpu_partial_sum,
double* d_gpu_sum)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    double sum = 0.0;
    for(long element = id; element < n; element += gridsize) {
        sum += d_C[element];
    }
    d_gpu_partial_sum[id]=sum;
    __syncthreads();

    if(threadIdx.x==0) {
        d_gpu_sum[blockIdx.x] = 0.0;
        for(int j = 0; j < blockDim.x; j++){
            d_gpu_sum[blockIdx.x] +=
                d_gpu_partial_sum[ blockIdx.x*blockDim.x+j];
        }
    }
}
```



Remember Listing2-6: multiple elements per thread

```
__global__ void reduction(long n, int gridsize, float* d_C, double* d_gpu_partial_sum,
double* d_gpu_sum)
{
    long id = blockIdx.x*blockDim.x+threadIdx.x;
    double sum = 0.0;
    for(long element = id; element < n; element += gridsize) {
        sum += d_C[element];
    }
    d_gpu_partial_sum[id]=sum;
    __syncthreads();

    if(threadIdx.x==0) {
        d_gpu_sum[blockIdx.x] = 0.0;
        for(int j = 0; j < blockDim.x; j++){
            d_gpu_sum[blockIdx.x] +=
                d_gpu_partial_sum[ blockIdx.x*blockDim.x+j];
        }
    }
}
```



We need to extend this idea to the computation.

Testing a faster Reduction in CUDA: Listing2-6

```
[jla061@birget Listings2]$ nvcc Listing2-6-fastreductionvecmult.cu -o
fastreductionvecmult -Xcompiler -fopenmp
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000
gpu sum = 18446744069414598656.0000000000,steps 5000000000 compute time 0.1194 s
reduction time 0.02840 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024
gpu sum = 18446744069414598656.0000000000,steps 5000000000 compute time 0.0394 s
reduction time 0.02848 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024 128
gpu sum = 18446744069414594560.0000000000,steps 5000000000 compute time 0.0394 s
reduction time 0.01336 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024 64
gpu sum = 18446744069414594560.0000000000,steps 5000000000 compute time 0.0391 s
reduction time 0.01316 s
[jla061@birget Listings2]$ ./fastreductionvecmult 5000000000 1024 64 64
gpu sum = 18446744069414584320.0000000000,steps 5000000000 compute time 0.0394 s
reduction time 0.01683 s
```

Reduction still a bit slow since we need to read the data again.



Using Shared Memory: Listing2-7

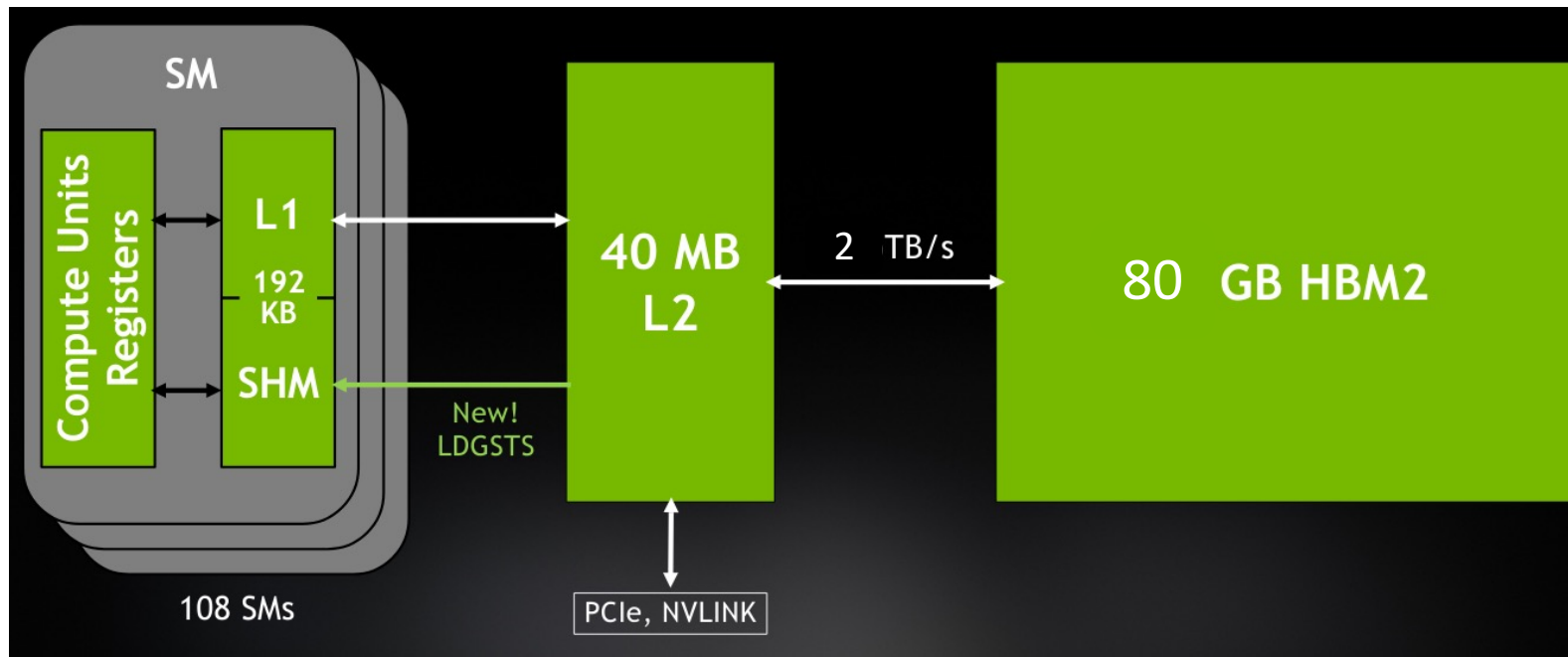
```
__global__ void fusedgpuvecmul(long n, float* d_A, float* d_B, double* d_gpu_sum)
{
    __shared__ double C_temp[1024];

    long id = blockIdx.x*blockDim.x+threadIdx.x;
    if(id<n)
        C_temp[threadIdx.x] = d_A[id]*d_B[id];
    __syncthreads();
    if(threadIdx.x==0) {
        d_gpu_sum[blockIdx.x] = 0.0;
        for(int j = 0; j < blockDim.x; j++){
            d_gpu_sum[blockIdx.x] += C_temp[j];
        }
    }
}

__global__ void reductionFinish(int blocks, double* d_final_gpu_sum, double* d_gpu_sum)
{
    for(long i = 0; i < blocks; i++){
        d_final_gpu_sum[0] += d_gpu_sum[i];
    }
}
```



GPU Memory



- Device memory is fairly large, but high latency
- L2 caches device memory. Not controllable.
- L1/Shared memory can be controlled by programmer. Visible within the block.



Testing the Shared Memory Reduction: Listing2-7

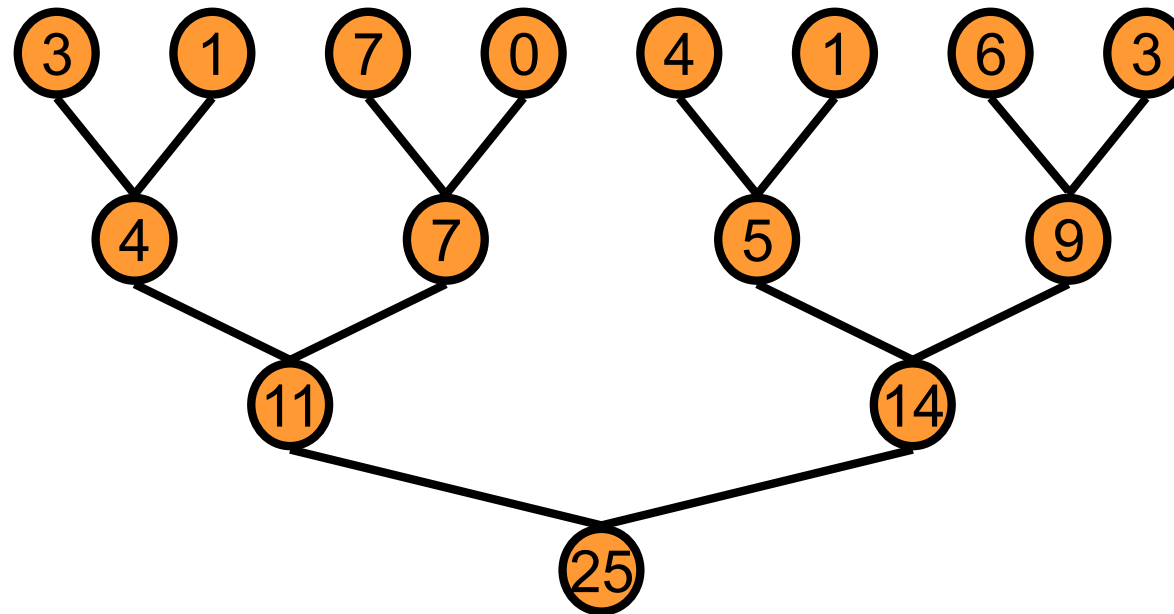
```
[jla061@birget Listings2]$ ./fusedreductionvecmult 10000000000 256
gpu sum = 38881772600738807808.0000000000,steps 10000000000 compute time 0.1238 s
reduction time 3.94313 s
[jla061@birget Listings2]$ ./fusedreductionvecmult 10000000000 512
gpu sum = 38881772600109826048.0000000000,steps 10000000000 compute time 0.1122 s
reduction time 1.92843 s
[jla061@birget Listings2]$ ./fusedreductionvecmult 10000000000 1024
gpu sum = 38881772599220305920.0000000000,steps 10000000000 compute time 0.3519 s
reduction time 0.96158 s
```

Can run larger instances again, but now the reduction is sequential and thus very slow. How to fix that?

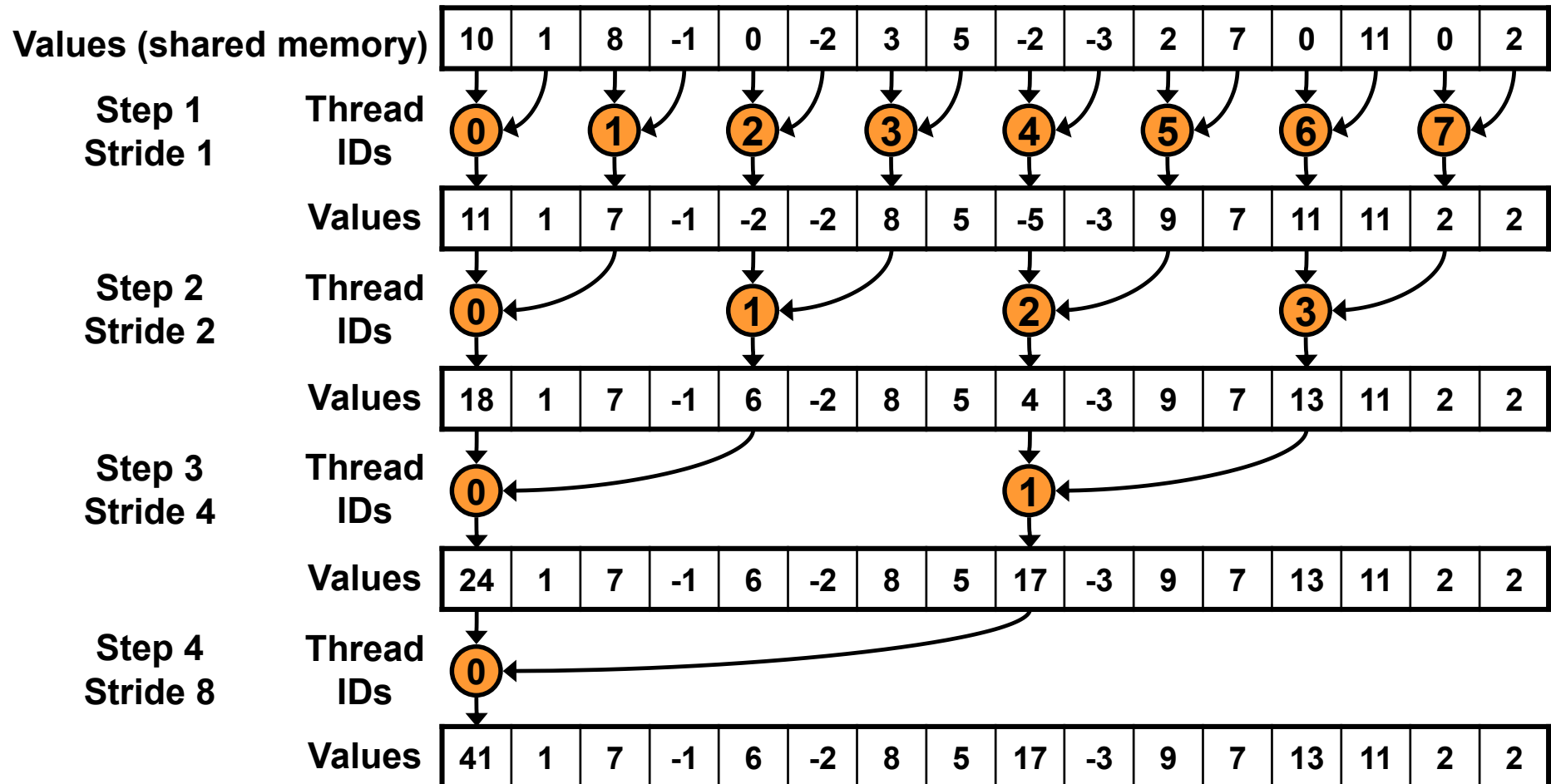


Reduction in CUDA

- We want to get away from summing in individual threads
- We want to maximize parallelism
- Classic idea: tree reduction

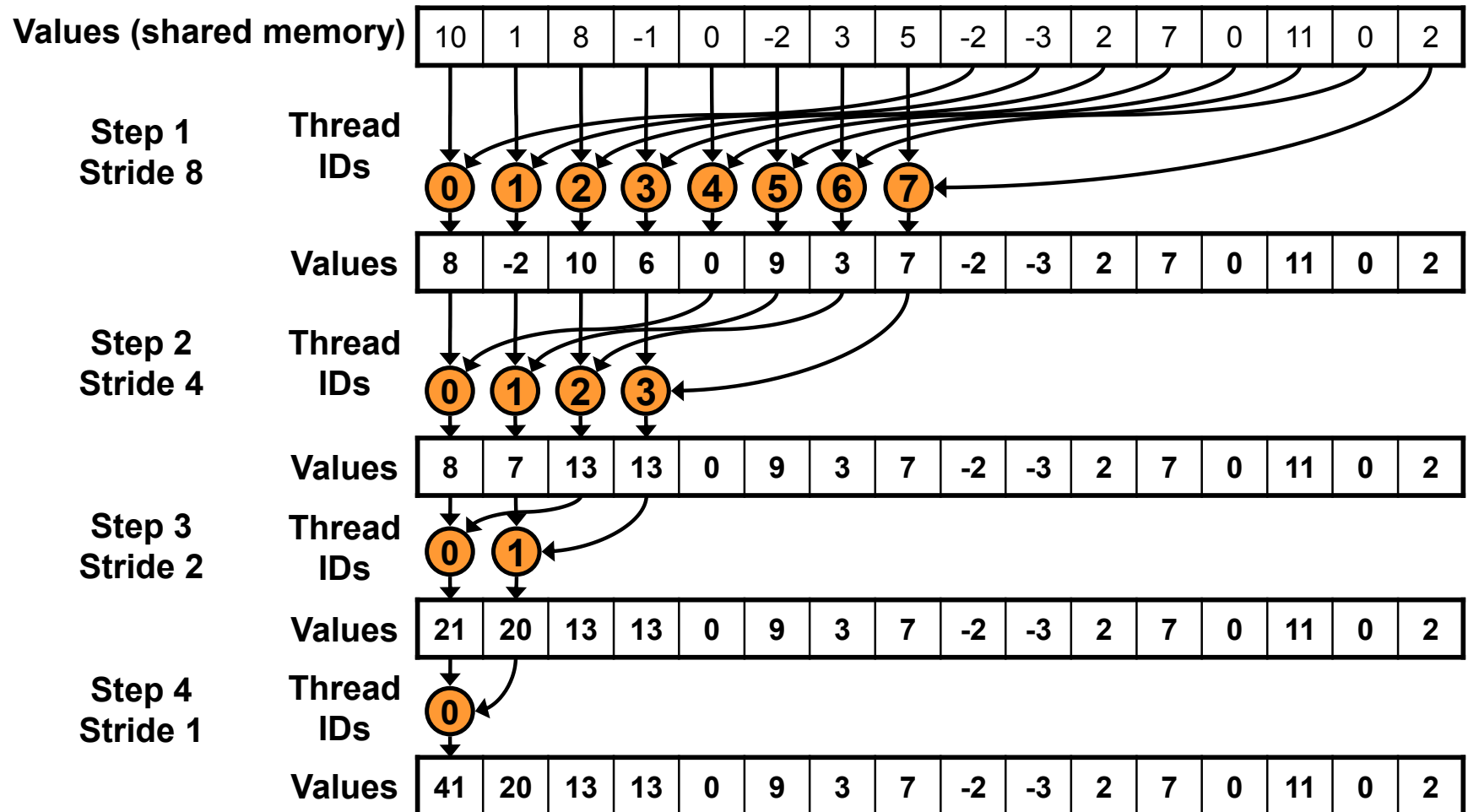


Reduction in CUDA



Harris, M. (2007). Optimizing parallel reduction in CUDA. *Nvidia developer technology*, 2(4), 70.

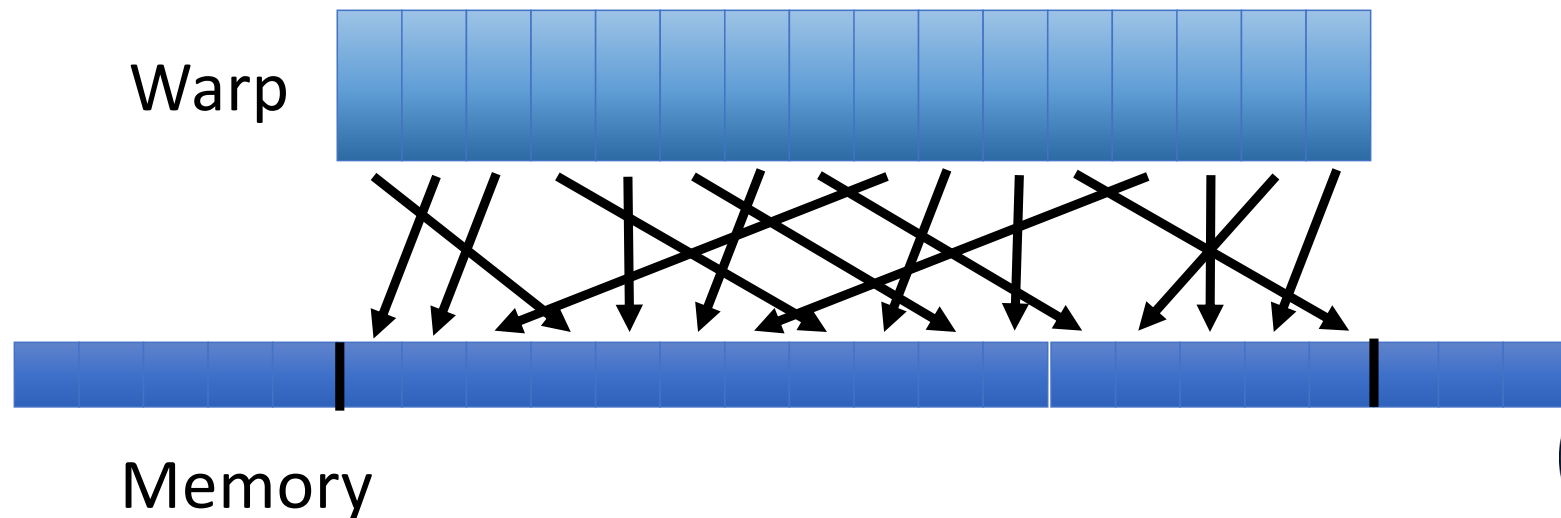
Sequential addressing improves performance



Harris, M. (2007). Optimizing parallel reduction in CUDA. *Nvidia developer technology*, 2(4), 70.

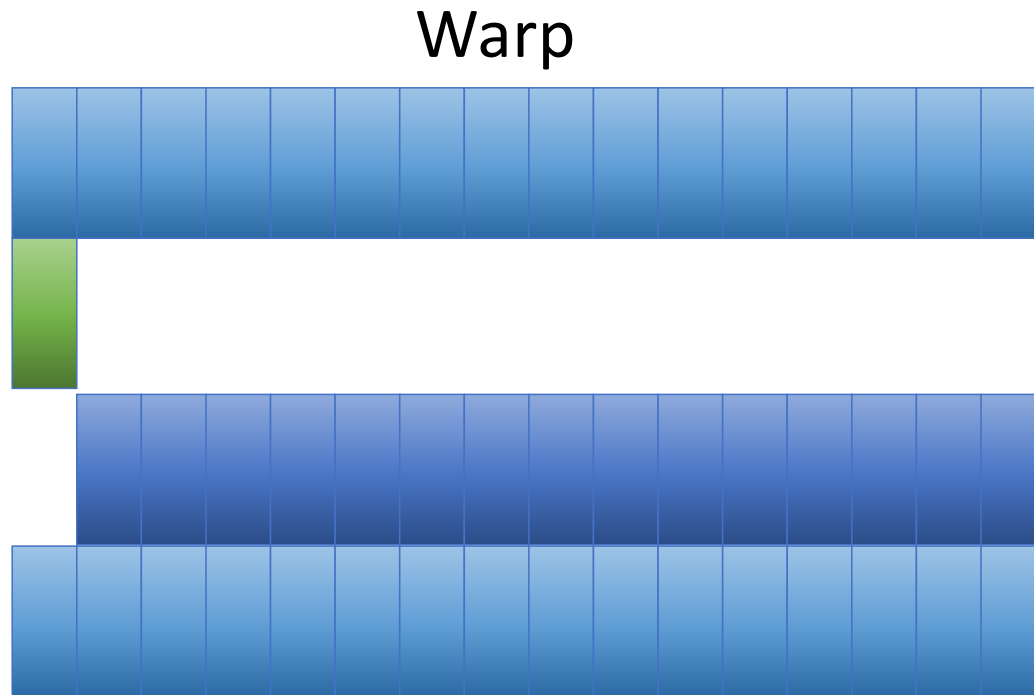
SIMT Problems: Remember memory Coalescing

A warp of 32 threads must read 32 contiguous elements from an array to get the maximum memory bandwidth, although elements can be swapped between the threads.



SIMT Problems: Warp Divergence

```
int index =  
threadIdx.x  
+ blockIdx.x  
* blockDim.x;  
if(index == 0)  
c[index]=a[index]*2;  
else  
c[index] = a[index]  
+ b[index];  
return 0;
```

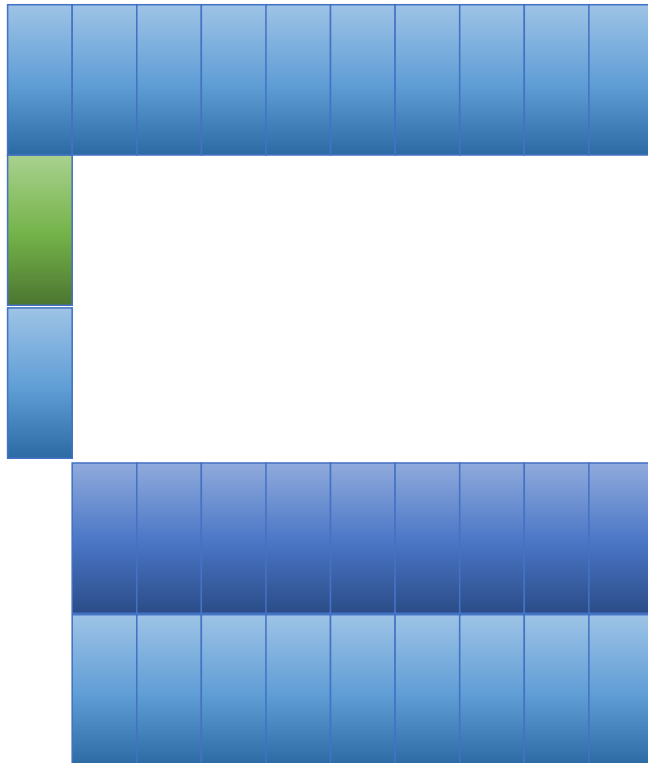


`else` instructions or other branching within the warp are expensive. Avoid if possible.

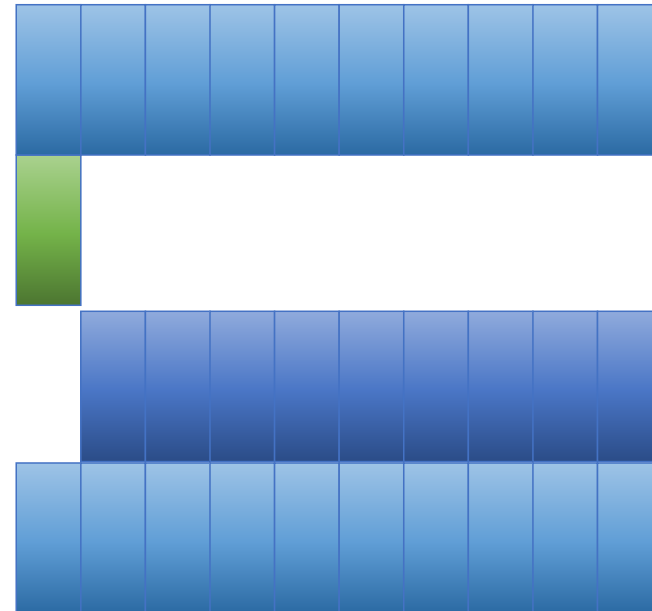


SIMT Problems: Warp Divergence

Pre-Volta



Volta, Ampere, etc.



Somewhat improved in Volta generation, but still SIMT (SIMD).

Synchronization cost

```
__global__ void reduce0(int *g_idata, int *g_odata) {  
    extern __shared__ int sdata[];  
    // each thread loads one element from global to shared mem  
    unsigned int tid = threadIdx.x;  
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
    sdata[tid] = g_idata[i];  
    __syncthreads();  
    // do reduction in shared mem  
    for(unsigned int s=1; s < blockDim.x; s *= 2) {  
        if (tid % (2*s) == 0) {  
            sdata[tid] += sdata[tid + s];  
        }  
        __syncthreads();  
    }  
}
```

This is actually
quite expensive



Synchronization cost

```
if (id < 32) {  
    // syncwarps required for devices of CC >= 7.0  
    if(id < 32) { s[id] += s[id + 32]; __syncwarp();  
    if(id < 16) s[id] += s[id + 16]; __syncwarp();  
    if(id < 8) s[id] += s[id + 8]; __syncwarp();  
    if(id < 4) s[id] += s[id + 4]; __syncwarp();  
    if(id < 2) s[id] += s[id + 2]; __syncwarp();  
    if(id < 1) s[id] += s[id + 1]; __syncwarp();  
}
```

Much faster at
warp level

- Recent GPUs add more warp-level instructions
- Possible to implement reduction at warp level only

